

# Integrate an External SOAP Partner

## Goal

Integrate **one external SOAP partner** into CampusEats at the contract level: author the partner's WSDL, form the SOAP call by hand, and describe how you would discover it. **This is design-time only — no running server and no live gateway. You are not building one of your own services here.**

## Approach

Apply the **Tutorial 3 method** to a partner edge of **your own** CampusEats: **read / author the WSDL → form the SOAP request, response and fault → show the HTTP binding → describe discovery**. Keep it small but complete. Discovery is done in **modern form** — a catalogue / registry entry; you do **not** run a UDDI server.

## Tasks

- Pick the partner edge and justify SOAP.** Choose one external partner CampusEats must integrate — a payment gateway, a logistics/delivery partner, or an SMS gateway. In 3–4 sentences, explain why **that** edge is SOAP while the rest of CampusEats is REST (strong contract, message-level security, transactions).  
**Submit** — a short “Context” section in *integration.pdf* naming the partner and the one operation you'll integrate.
- Author the partner's WSDL.** Write a small but complete WSDL for the partner service with **at least one operation** (e.g. charge, assignRider, or sendSMS). It must contain all six elements: types, message, portType, binding (SOAP over HTTP), and service/port with a real-looking endpoint URL.  
**Submit** — *partner.wsdl* — the contract your CampusEats consumer binds to.
- Form the SOAP messages by hand.** Write three envelopes that match your WSDL element-for-element: the **request** (credentials in the Header, the call in the Body), the success **response**, and one `soap:Fault` for a realistic failure (e.g. declined / unavailable).  
**Submit** — *soap-request.xml*, *soap-response.xml*, *soap-fault.xml*.
- Show the HTTP binding.** Show the HTTP POST that carries the request: method, Host, Content-Type, the SOAPAction value from your binding, and the endpoint URL from your service/port.  
**Submit** — the HTTP request block in *integration.pdf*.
- Describe discovery (modern form).** Explain how CampusEats would obtain this WSDL — a direct URL, or a lookup — and write a one-record **catalogue / registry entry**: business, service, endpoint, and the WSDL it speaks (a tModel-style pointer). No live UDDI server is required.  
**Submit** — a “Discovery” section with the registry entry in *integration.pdf*.
- Map the fault into your own contract.** Show how the partner's fault becomes the error your Assignment 2 contract promised (e.g. the partner's `card_declined` → your `placeOrder` “payment declined”). The partner's vocabulary must not leak to your students.  
**Submit** — a short “Fault mapping” note in *integration.pdf*.

## What to submit

Commit all of the following into your **campuseats** repository, and submit the link:

- *integration.pdf* — describe context, HTTP binding, discovery + registry entry, and fault mapping (Tasks 1, 4, 5, 6).
- *partner.wsdl* — the partner contract (Task 2).
- *soap-request.xml*, *soap-response.xml*, *soap-fault.xml* — the messages (Task 3).
- *integration.pdf* — should indicate the TeamID, Roll/Name of all members.
- One Zip file with the above three documents, to be uploaded by Team Leader.